Sabanci University

Operating Systems

CS 307

Programming Assignment - 4: LC-3 Language to Write Programs for the LC-3 Virtual Memory Simulator

Release Date: 12 December 2025
Deadline: 22 December 2025, 00:00

1 Introduction

For the virtual memory simulator that you are going to implement in the Programming Assignment - 4 (PA4), we are introducing a new language, the LC-3 Language, which you can use to write your own programs to run on the simulator.

LC-3 Language supports up to 4 registers (R1, R2, R3, and R4), which can be called the **value registers**, to store values in a variable-like manner, and the actual **variables** that are stored in the heap segment of the simulator, similar to the variables in the actual programming languages for other platforms and architectures.

Note: You can only store integer values within the range $[-4080, 3825]$ in the value registers and variables, and all the integers in the LC-3 architecture are assumed to be signed integers (Source: The Wikipedia page for the LC-3 architecture).

2 Language Syntax

2.1 Basic Rules

The delimiter for statements in the LC-3 Language is `;` as in C and C++. After the `if`, `else`, and `while` statements, you should use `:` to start the relevant block as in Python, and the if-else blocks and while blocks have to be concluded with an `end` keyword as it is going to be explained in the subsequent sections, and presented in the language specification.

LC-3 Language also supports comments added into the codes as single-line comments; you can use the characters `//` to begin your comment sections whether inline or covering an entire line.

Note: Multi-line comments as blocks on their own are not supported in LC-3 Language.

In order to write your codes in LC-3 Language and compile them for the simulator, you should perform the following steps:

- Write your codes into a text file with the extension `.lc3`, e.g., `filename.lc3`.

- In order to compile and assemble your code, run the following command (assuming you are using the CS 307 server):

```
$ ./lc3l filename.lc3
```

  - By this step, you will have generated the following files: `filename.asm`, which is the assembly code corresponding to your code, and the files `filename_code.obj` and `filename_heap.obj`, which you can pass to the simulator as the code segment and the heap segment of your program, respectively.

2.2 Assignment Operation

The assignment operator in LC-3 Language is `=` as in most programming languages; the left-hand side of an assignment can be a variable or a value register, while the right-hand side can be a value register, a variable, a constant value, and the supported arithmetic expressions containing the use of the aforementioned tokens.

Below, you can find examples for the assignment operation in LC-3 Language:

```
R1 = R2;
```

```
R1 = var;
```

```
R1 = 2;
```

```
R1 = R2 + 2;
```

```
R1 = 2 + R2;
```

```
R1 = var + 2;
```

```
R1 = 2 + var;
```

```
R1 = 3 + 2;
```

```
R1 = R2 - 2;
```

```
R1 = 2 - R2;
```

```
R1 = var - 2;
```

```
R1 = 2 - var;
```

```
R1 = 3 - 2;
```

```
var1 = R2;
```

```
var1 = var2;
```

```
var1 = 2;
```

```
var1 = R2 + 2;
```

```
var1 = 2 + R2;
```

```
var1 = var2 + 2;
```

```
var1 = 2 + var2;
```

```
var1 = 3 + 2;
```

```
var1 = R2 - 2;
```

```
var1 = 2 - R2;
```

```
var1 = var2 - 2;
```

```
var1 = 2 - var2;
```

```
var1 = 3 - 2;
```

2.3 Arithmetic Operations

The supported arithmetic operations in LC-3 Language are `+` and `-`, which can be utilized within the arithmetic expressions containing value registers, variables, and constant variables; you can use arithmetic expressions at the right-hand side of the assignments in LC-3 Language.

Below, you can find examples for the arithmetic expressions supported in LC-3 Language:

```
R1 + 2;
```

```
2 + R1;
```

```
var + 2;
```

```
2 + var;
```

```
3 + 2;
```

```
R1 - 2;
```

```
2 - R1;
```

```
var - 2;
```

```
2 - var;
```

```
3 - 2;
```

2.4 Comparisons

LC-3 Language supports the following comparisons between value registers, variables, and constants:

- `==` for the check of **equals**
- `!=` for the check of **not equals**

- `<` for the check of **strictly less than**
- `<=` for the check of **less than or equal to**
- `>` for the check of **strictly greater than**
- `>=` for the check of **greater than or equal to**

Note: You can only use comparisons to serve as conditions in the conditional expressions and loops in LC-3 Language, and inside parentheses `()`.

Below, you can find some examples for the comparisons supported in LC-3 Language:

```
(R1 == R2)
```

```
(R1 != var)
```

```
(var != R1)
```

```
(R1 > 5)
```

```
(5 < R1)
```

```
(var > 2)
```

```
(2 < var)
```

```
(3 >= 2)
```

```
(2 <= 3)
```

2.5 Conditional Expressions

Conditional expressions are supported by LC-3 Language as `if-else-end` blocks and `if-end` blocks depending on the comparisons; you can check some of the examples of conditional expressions in LC-3 Language below:

```
if (o2 >= 1):
    R1 = R1 + R4;
else:
    R1 = R1 - 5;
end
```

```
if (iteration < 6):
    iteration = iteration + 1;
end
```

Furthermore, LC-3 Language supports nested `if-else-end`/`if-end` blocks, some of whose examples are presented as follows:

```
if (value < threshold):
    value = value + 2;
    if (value > threshold):
        value = threshold;
    else:
        value = 0;
    end
else:
    value = value - 1;
end
```

```
if (value >= threshold):
    value = value - 1;
else:
    value = value + 2;
    if (value > threshold):
        value = threshold;
    else:
        value = 0;
    end
end
```

```
if (value >= threshold):
    value = value - 1;
else:
    value = value + 2;
    if (value > threshold):
        value = threshold;
    end
end
```

Note: The following kind of nested conditional expressions is not supported in the LC-3 Language (see the Dangling Else Problem if you are interested):

```
if (value < threshold):
    value = value + 2;
    if (value > threshold):
        value = threshold;
    end
else:
    value = value - 1;
end
```

2.6 Loops

LC-3 Language supports while loops through the `while-end` blocks, a basic example for which is available below (assume that the mentioned variables are defined earlier in the program):

```
while (iteration < limit):
    tempVar = currentSum;
    currentSum = currentSum + previousSum;
    previousSum = tempVar;
    iteration += 1;
end
```

In addition to the basic while loops, LC-3 Language supports nested while loops as well:

```
while (x != 0):
    inner = y;

    while (inner != 0):
        result = result + 1;
        inner = inner - 1;
    end

    x = x - 1;
end
```

Moreover, you can write a while loop inside a conditional expression, or vice versa:

```
if (x != 0):
    inner = y;

    while (inner != 0):
        result = result + 1;
        inner = inner - 1;
    end

    x = x - 1;
end
```

```
while (o1 != 0):
    if (o2 < 1):
        R1 = R1 + R4;
    else:
        R1 = R1 - 5;
    end

    o1 = o1 - 1;
end
```

Note: LC-3 Language does not support for loops.

2.7 Instructions Supported as Keywords

LC-3 Language supports direct invocation of the `yield` and `brk` instructions, which you are going to implement in PA4, within the source code. You can write the following statements in your codes to invoke these instructions directly:

```
YIELD; // The yield instruction as you are going to
   implement in tyld()
```

```
BRK(vpn, write, read, alloc); // The brk instruction
   as you are going to implement in tbrk()
```

- The `BRK` instruction statement takes the `vpn` value, which must be between 0 and 31 (both boundaries are included), `write` bit value, which must be 0 or 1 to disable/enable writing on the page, `read` bit value, which must be 0 or 1 to disable/enable reading of the page, and the `alloc` bit value, which must be 0 for deallocation and 1 for allocation.
- **Note:** You can specify `write` and `read` values for deallocation as 0 in the programs.

3 The lc3l Suite

The compiler and assembler suite for the LC-3 Language, `lc3l`, resides at the file `lc3l.c`, and it can be compiled using the Makefile, along with other files for PA4, provided in the bundle.

3.1 The Compiler in the Suite

The compiler part of `lc3l` takes an `.lc3` file, in which you had written your code, as its argument, and returns you two files:

- `filename.asm`: The file where your code in LC-3 Language is compiled by the compiler part of `lc3l` into the assembly language for the LC-3 architecture. This file is automatically going to be passed into the

assembler part of the suite, which is going to be explained in the next section.

- `filename_heap.obj`: The binary file which serves as the initial heap segment of your program and contains the initial values for the variables as you initialized them in your code; you are going to pass this file into the simulator, `vm`, as a command-line argument, which is explained in the PA4 document.

3.2 The Assembler in the Suite

The assembler part of `lc3l` takes an `.asm` file, which is generated by the compiler part of `lc3l`, as its argument, and returns you a single file:

- `filename_code.obj`: The binary file containing the code segment of your program; you are going to pass this file into the simulator, `vm`, as a command-line argument, which is explained in the PA4 document.

3.3 The Development of the Suite

The `lc3l` suite is developed using the `mpc` library, which is used to scan and parse the `.lc3` source files according to the grammar specified for the LC-3 Language. The files for the library, `mpc.c` and `mpc.h`, are also included as dependencies for the suite in the bundle.

4 Language Specification

The following contents are provided to you in case that you want to know which kinds of statements you are allowed to write in the LC-3 Language, or you are curious about the language specification of the LC-3 Language, and you can implement your own scanner and parser for the LC-3 Language using these specifications with `flex` and `bison`.

4.1 Scanner Specification for the LC-3 Language

```
%{
#include "lc3.tab.h"
%}

tREG            "R"[1-4]
tID             [a-zA-Z][a-zA-Z0-9]*

tNZINT          "-"?[1-9][0-9]*
tINT            ({tNZINT}|0)

tASSIGN         "="
tPLUSASSIGN     "+="
tMINUSASSIGN    "-="

tPLUS           "+"
tMINUS          "-"

tEQ             "=="
tNEQ            "!="
tLT             "<"
tGT             ">"
tLTE            "<="
tGTE            ">="

tLPR            "("
tRPR            ")"
tCOMMA          ","

tIF             "if"
tELSE           "else"
tWHILE          "while"
tEND            "end"
tBRK            "BRK"
tYIELD          "YIELD"

tCOLON          ":"
tSEMICOLON      ";"
```

```
%x SLCOMMENT
%%
<SLCOMMENT>"\n"          {
                           BEGIN(INITIAL);
                         }
<SLCOMMENT>.             ;

{tASSIGN}                {return tASSIGN;}
{tPLUSASSIGN}            {return tPLUSASSIGN;}
{tMINUSASSIGN}           {return tMINUSASSIGN;}

{tPLUS}                  {return tPLUS;}
{tMINUS}                 {return tMINUS;}

{tLTE}                   {return tLTE;}
{tGTE}                   {return tGTE;}
{tNEQ}                   {return tNEQ;}
{tEQ}                    {return tEQ;}
{tLT}                    {return tLT;}
{tGT}                    {return tGT;}

{tLPR}                   {return tLPR;}
{tRPR}                   {return tRPR;}
{tCOMMA}                 {return tCOMMA;}

{tIF}                    {return tIF;}
{tELSE}                  {return tELSE;}
{tWHILE}                 {return tWHILE;}
{tEND}                   {return tEND;}
{tBRK}                   {return tBRK;}
{tYIELD}                 {return tYIELD;}

{tREG}                   {return tREG;}
{tID}                    {return tID;}

{tINT}                   {return tINT;}

{tCOLON}                 {return tCOLON;}
{tSEMICOLON}             {return tSEMICOLON;}
```

```
"//"                    {BEGIN(SLCOMMENT);}

[ \t\r\n]               ;
%%
```

4.2 Parser Specification for the LC-3 Language

```
%{
#include <stdio.h>
int yylex(void);

void yyerror(const char *msg) {
  printf("%s\n", msg);
}
%}
%define parse.error verbose
%token tREG tID tINT tASSIGN tPLUSASSIGN tMINUSASSIGN
   tPLUS tMINUS tEQ tNEQ tLT tGT tLTE tGTE tLPR tRPR
   tCOMMA tIF tELSE tWHILE tEND tBRK tYIELD tCOLON
   tSEMICOLON
%start lc3
%%
lc3:                stmts;
stmts:              stmt stmts
                  | /*null*/
                  ;

stmt:               assignment
                  | selfarithmetic
                  | matched_if
                  | unmatched_if
                  | while_stmt
                  | brk_stmt
                  | yield_stmt
                  ;

assignment:         tREG tASSIGN arithmetic tSEMICOLON
```

```
                  | tID tASSIGN arithmetic tSEMICOLON
                  ;

arithmetic:         term tPLUS term
                  | term tMINUS term
                  | term
                  ;

term:               tREG
                  | tID
                  | tINT
                  ;

selfarithmetic:     tREG tPLUSASSIGN term tSEMICOLON
                  | tREG tMINUSASSIGN term tSEMICOLON
                  | tID tPLUSASSIGN term tSEMICOLON
                  | tID tMINUSASSIGN term tSEMICOLON
                  ;

matched_if:         tIF tLPR boolean tRPR tCOLON
   matched_stmts tELSE tCOLON unmatched_stmts tEND;

matched_stmts:      matched_stmt matched_stmts
                  | /*null*/
                  ;

matched_stmt:       assignment
                  | selfarithmetic
                  | matched_if
                  | while_stmt
                  | yield_stmt
                  ;

unmatched_if:       tIF tLPR boolean tRPR tCOLON
   unmatched_stmts tEND;

while_stmt:         tWHILE tLPR boolean tRPR tCOLON
   unmatched_stmts tEND;

unmatched_stmts:    unmatched_stmt unmatched_stmts
```

```
                    | /*null*/
                    ;

unmatched_stmt:       assignment
                    | selfarithmetic
                    | matched_if
                    | unmatched_if
                    | while_stmt
                    | yield_stmt
                    ;

boolean:              term tLTE term
                    | term tGTE term
                    | term tNEQ term
                    | term tEQ term
                    | term tLT term
                    | term tGT term
                    ;

brk_stmt:             tBRK tLPR tINT tCOMMA tINT tCOMMA
   tINT tCOMMA tINT tRPR tSEMICOLON;

yield_stmt:           tYIELD tSEMICOLON;

%%
int main() {
  return yyparse();
}
```

4.3 (Extra) How to Generate the Parser from the Specification

If you wish to generate a parser for the LC-3 Language, you should run the following commands on your terminal:

```
flex lc3.flx # Generates lex.yy.c

bison -d lc3.y # Generates lc3.tab.h and lc3.tab.c
```

```
gcc -o lc3p lc3.tab.c lex.yy.c -lfl # Generates the
   parser executable
```

5 Extra Resources

If you are curious about the computer architectures and assembly instructions, you can check, in the scope of LC-3 architecture, which is a computer architecture designed for educational purposes, and the PA4 as a kind of introduction, the following resources:

- The slides from University of Texas at Austin on the LC-3 architecture.
- The Wikipedia page for the LC-3 architecture.
- The blog by Andrei Ciobanu on the virtual machine which constitutes the basis of PA4 (link is already provided in the PA4 document).